NUMPY

Quick Reference

A practical revision guide based on the two uploaded class notebooks

What this guide covers

- NumPy arrays (ndarray), dimensions, shape, size and data types
- Creating arrays from Python data, fixed values, random values and ranges
- Indexing, slicing, advanced indexing and iteration
- Reshaping, flattening, concatenating, transposing and splitting
- Arithmetic, mathematical, logical and comparison operations
- Sorting, searching, counting and unique values
- Loading tabular data, descriptive statistics and correlation
- Basic matrix operations from the class examples

Getting started: NumPy must be installed in the Python environment used by Jupyter. Most examples begin with import numpy as np.

```
import numpy as np
```

1. Python list vs NumPy array

A Python list repeats when multiplied by an integer. A NumPy array performs element-wise arithmetic, which is useful for numerical work.

```
# Python list
arr = [1, 2, 3]
print(arr * 5)                # repeats the list

# List comprehension: multiply each element
arr = [x * 5 for x in arr]
print(arr)

# NumPy array: multiplication is element-wise
arr1 = np.array([1, 2, 3])
arr1 = arr1 * 5
print(arr1)
```

```
[1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3]
[5, 10, 15]
[ 5 10 15]
```

2. ndarray and dimensions

The main NumPy data structure is the n-dimensional array, usually called an ndarray. A 1D array is like a list, a 2D array is arranged in rows and columns, and a 3D array can be viewed as a stack of 2D arrays.

```
arr1 = np.array([1, 2, 3, 4, 5])       # 1D

arr2 = np.array([
    [1, 2, 3],                          # row 1
    [4, 5, 6]                           # row 2
])                                      # 2D

arr3 = np.array([
    [[1, 2, 3], [4, 5, 6]],             # first 2D layer
    [[1, 2, 3], [4, 5, 6]],             # second layer
    [[1, 2, 3], [4, 5, 6]]              # third layer
])                                      # 3D
```

Note: For a regular array, every row/layer should have a consistent structure and compatible data.

3. Array attributes

Attributes describe an array. They do not call a function, so write them without parentheses.

```
print(arr1.ndim)   # number of dimensions
print(arr1.shape)  # length along each dimension
print(arr1.dtype)  # element data type
print(arr1.size)   # total number of elements
```

```
# For arr1 = [1, 2, 3, 4, 5]
# ndim  -> 1
# shape -> (5,)
# size  -> 5
```

Note: For arr2 with 2 rows and 3 columns, shape is (2, 3), ndim is 2 and size is 6.

4. ndarray data types

NumPy arrays normally store one common data type. When values have mixed types, NumPy may convert them to a common type (called type promotion or upcasting).

```
arr = np.array([1, 2, 3])
print(arr.dtype)                # integer type

arr = np.array([1.5, 2.4, 3.2])
print(arr.dtype)                # floating-point type

arr = np.array([1, 2, 3.2])
print(arr.dtype)                # integers are promoted to float

arr = np.array([1, 2, 3.2, "hello"])
print(arr.dtype)                # values become strings
```

Choose or convert a data type

```
arr = np.array([1, 2, 300], dtype=np.int16)
print(arr.dtype)

arr = arr.astype(np.int32)
print(arr.dtype)
```

Note: A conversion such as np.array([1, 2, 3.2, 'hello'], dtype=np.float64) raises an error because 'hello' cannot be converted to a number. Integer types such as int16 have a limited numeric range; choose a type that can hold your values.

5. Creating arrays from existing Python data

np.array() can create an ndarray from common Python collections.

From a list or nested list

```
lst = [10, 20, 30, 40, 40.5]
arr = np.array(lst)
print(arr)

matrix = [[1, 2, 3],
          [4, 5, 6]]
arr = np.array(matrix)
print(arr)
print(type(arr), arr.dtype, arr.shape, arr.ndim)
```

From a tuple or set

```
tpl = (10, 20, 30)
arr = np.array(tpl, dtype=np.int32)
arr = arr.astype(np.int16)
print(arr)

st = {1, 2, 3}
arr = np.array(list(st))        # convert set to list first
print(arr)
```

Note: A set is unordered. Do not rely on its array elements appearing in a particular order.

From a dictionary

```
dc = {"a": 10, "b": 20, "c": 30}

keys = dc.keys()
values = dc.values()
items = dc.items()

keys_arr = np.array(list(keys))
values_arr = np.array(list(values))
items_arr = np.array(list(items))

print(keys_arr)      # dictionary keys
print(values_arr)    # dictionary values
print(items_arr)     # key-value pairs
```

Note: Dictionary keys and values are converted separately. Converting items creates paired key-value rows; because keys and values have different types, NumPy may store the resulting values as strings.

6. Creating arrays from scratch

np.zeros() and np.zeros_like()

Create an array filled with zeros. The *_like version copies the shape of an existing array.

Syntax

```
np.zeros((rows, columns), dtype=...)
np.zeros_like(existing_array)
```

Example

```
a = np.zeros((2, 3), dtype=np.int8)
b = np.zeros_like(arr3)
print(a)
print(b.shape)
```

Note: Use a tuple for a multidimensional shape, such as (2, 3).

np.ones() and np.ones_like()

Create an array filled with ones.

Syntax

```
np.ones((rows, columns), dtype=...)
np.ones_like(existing_array)
```

Example

```
a = np.ones((3, 4), dtype=np.int8)
b = np.ones_like(arr3)
print(a)
print(b.shape)
```

np.empty() and np.empty_like()

Create an array with a requested shape without initializing its values.

Syntax

```
np.empty((rows, columns), dtype=...)
np.empty_like(existing_array)
```

Example

```
a = np.empty((4, 3), dtype=np.int8)
b = np.empty_like(arr3)
print(a.shape, b.shape)
```

Note: The values in an empty array are uninitialized and may look random. Fill the array before using its values.

np.full() and np.full_like()

Create an array filled with one chosen value.

Syntax

```
np.full((rows, columns), fill_value)
np.full_like(existing_array, fill_value, dtype=...)
```

Example

```
a = np.full((4, 3), np.inf)
b = np.full_like(arr3, np.inf, dtype=np.float64)
print(b)
print(b.dtype)
```

Note: np.inf represents positive infinity. Use a floating-point dtype when storing infinity.

7. Creating arrays with random values

np.random.rand()

Generate random floating-point values from the interval [0, 1). Pass each dimension as a separate argument.

Syntax

```
np.random.rand(rows, columns)
```

Example

```
arr = np.random.rand(2, 3)
print(arr)
print(arr.shape)
```

Note: The exact values change each time you run the cell.

np.random.randint()

Generate random integers from start (inclusive) to end (exclusive).

Syntax

```
np.random.randint(start, end, size=(rows, columns))
```

Example

```
arr = np.random.randint(1, 10, size=(2, 3))
print(arr)
```

Note: The upper bound is excluded, so this example can generate 1 through 9.

np.random.uniform()

Generate random floating-point values from a uniform range.

Syntax

```
np.random.uniform(start, end, size=(...))
```

Example

```
arr = np.random.uniform(50, 100, size=(3, 3, 4))
print(arr.shape)
```

Note: The generated values are at least 50 and less than 100.

8. Array creation with range functions

np.arange()

Create evenly spaced values using a start, stop and step. The stop value is excluded, similar to Python range().

Syntax

```
np.arange(start, stop, step)
```

Example

```
arr = np.arange(1, 10, 1)
mat = arr.reshape(3, 3)
print(arr)
print(mat)
```

Note: The requested reshape must preserve the total number of elements.

np.linspace()

Create a fixed number of evenly spaced values between start and stop. The endpoint is included by default.

Syntax

```
np.linspace(start, stop, num)
```

Example

```
arr = np.linspace(0, 4, 6)
print(arr)
```

Expected output

```
[0.  0.8 1.6 2.4 3.2 4. ]
```

np.logspace()

Create values that are evenly spaced on a logarithmic scale. The start and stop are exponents; base defaults to 10.

Syntax

```
np.logspace(start, stop, num, base=10)
```

Example

```
arr = np.logspace(0, 4, 6, base=2)
print(arr)
```

Note: With base=2, this produces powers of 2 from 2^0 to 2^4, with six logarithmically spaced values.

9. Matrix creation helpers (class bonus)

np.diag()

Create a square diagonal matrix from a 1D sequence. The given values appear on the main diagonal.

Syntax

```
np.diag([value1, value2, ...])
```

Example

```
diagonal_matrix = np.diag([1, 2, 3, 4])
print(diagonal_matrix)
```

np.eye()

Create a 2D array with ones on a selected diagonal and zeros elsewhere.

Syntax

```
np.eye(rows, columns=None, k=0)
```

Example

```
identity_mat = np.eye(4)
mat = np.eye(3, 4)
upper_diagonal = np.eye(3, 4, 1)
print(identity_mat)
print(upper_diagonal)
```

Note: k=0 selects the main diagonal; k=1 selects the diagonal above it. The default number of columns equals the number of rows.

10. Indexing and slicing

1D indexing and slicing

Indexing selects one element. Slicing selects a range. Python indexing starts at 0 and the stop index is excluded.

```
arr1 = np.array([10, 20, 30, 40, 50])

print(arr1[0])       # first element: 10
print(arr1[-1])      # last element: 50
print(arr1[1:4])     # elements at indexes 1, 2, 3
print(arr1[::2])     # every second element
```

Copy a slice

```
arr1_mod = arr1[1:4].copy()
arr1_mod[0] = 200

print(arr1_mod)
print(arr1)
```

Note: A basic NumPy slice is usually a view of the original array. Use .copy() when you want an independent array that can be changed without changing the original.

2D indexing and slicing

```
arr2 = np.array([[1, 2, 3],
                 [4, 5, 6]])

print(arr2[0, 2])          # row 0, column 2
arr2[0, 2] = 100            # update one element

row_0 = arr2[0:1, :]       # first row, kept 2D
row_1 = arr2[1:, :]        # second row, kept 2D

col_0 = arr2[:, 0:1]       # first column, kept 2D
col_1 = arr2[:, 1:2]       # second column, kept 2D

portion = arr2[:, 0:2]     # all rows, first two columns
```

Note: General 2D slice form: array[row_start:row_stop:row_step, column_start:column_stop:column_step]. Use array[row, column] for a single element.

11. Advanced indexing

Fancy indexing (select chosen positions)

```
values = np.array([10, 20, 30, 40])
selected = values[[0, 3, 1]]
print(selected)             # [10 40 20]
```

Select paired row/column positions

```
mat = np.array([[10, 20, 30],
                [40, 50, 60]])

selected = mat[[0, 1], [1, 2]]
print(selected)             # [20 60]
```

Note: The two index lists select paired coordinates: (row 0, column 1) and (row 1, column 2). They do not select a rectangular submatrix.

Boolean indexing and conditional replacement

```
arr2 = np.array([[1, 2, 3],
                 [4, 5, 6]])

print(arr2[arr2 > 3])       # elements greater than 3
arr2[arr2 > 3] = 0          # replace those elements with 0
print(arr2)
```

Note: The condition must refer to the array being indexed. The original notebook used arr > 1 while intending to filter arr2; this has been corrected to arr2 > 3.

12. Iterating through an array

np.nditer() visits each element of an array, including multidimensional arrays.

```
arr = np.array([[10, 20, 30],
                [100, 200, 300]])

for value in np.nditer(arr):
    print(value)
```

Note: The notebook also iterated over arr3. The example above is self-contained and demonstrates the same method.

13. Reshaping and flattening

reshape()

Change the shape of an array without changing its total number of elements.

Syntax

```
array.reshape(rows, columns)
```

Example

```
arr = np.random.randint(1, 100, size=(10, 5))
b = arr.reshape(5, 10)
print(b.shape)
```

Expected output

```
(5, 10)
```

Note: The total element count must remain the same: 10 × 5 = 5 × 10 = 50.

flatten()

Convert a multidimensional array into a 1D copy, using row-wise order by default.

Syntax

```
array.flatten()
```

Example

```
flat = b.flatten()
print(flat.ndim)
```

Expected output

```
1
```

np.ravel()

Return a flattened 1D view when possible (otherwise a copy). Choose row-wise or column-wise order.

Syntax

```
np.ravel(array, order='C')   # row-wise
np.ravel(array, order='F')   # column-wise
```

Example

```
row_wise = np.ravel(b, order="C")
column_wise = np.ravel(b, order="F")
```

Note: C means row-major order; F means column-major order. Use flatten() when you specifically need an independent copy.

14. Concatenating, transposing and splitting

np.concatenate()

Join arrays along an existing axis.

Syntax

```
np.concatenate((array1, array2), axis=0)  # add rows
np.concatenate((array1, array2), axis=1)  # add columns
```

Example

```
a = np.array([[1, 2, 3],
              [4, 5, 6]])
b = np.array([[7, 8, 9],
              [10, 11, 12]])

con_row = np.concatenate((a, b), axis=0)
con_col = np.concatenate((a, b), axis=1)
print(con_row.shape)        # (4, 3)
print(con_col.shape)        # (2, 6)
```

Note: For axis=0, the number of columns must match. For axis=1, the number of rows must match. The original notebook's column-wise concatenation of shapes (2, 3) and (1, 3) correctly raises a shape-mismatch error.

Transpose (.T)

Swap rows and columns of a 2D array.

Syntax

```
transposed = array.T
```

Example

```
mat = np.array([[10, 20, 30],
                [30, 40, 50]])
print(mat.T)
```

Expected output

```
[[10 30]
 [20 40]
 [30 50]]
```

np.array_split() and np.split()

Split an array into sections. array_split() allows sections of unequal sizes; split() requires equal division.

Syntax

```
np.array_split(array, number_of_sections)
np.split(array, number_of_equal_sections)
```

Example

```
a = np.arange(1, 11)

parts = np.array_split(a, 3)
print(parts)

equal_parts = np.split(np.arange(1, 10), 3)
print(equal_parts)
```

Note: The original np.split(a, 3) example used 10 elements, which cannot be divided into 3 equal sections and raises ValueError. Use np.array_split() for that case, or use 9 elements with np.split().

15. Arithmetic operators and mathematical functions

NumPy arithmetic is vectorized: operations are applied element by element, which avoids writing a Python loop for simple array calculations.

```
x = np.array([10, 8, 30, 100])
y = np.array([2, 3, 4, 5])

print(x + y)     # addition
print(x - y)     # subtraction
print(x * y)     # multiplication
print(x / y)     # division
print(x % y)     # remainder
```

Note: Both arrays in this example have the same shape, so corresponding elements are paired.

Trigonometry and angle conversion

```
x = np.array([0, 1, 2])

print(np.sin(x))
print(np.cos(x))
print(np.tan(x))

degrees = np.rad2deg(x)   # radians to degrees
radians = np.deg2rad(degrees)  # degrees to radians
```

Note: NumPy's sin(), cos() and tan() expect angles in radians, not degrees.

Logarithms and square root

```
x = np.array([10, 8, 16, 100])

print(np.log10(x))  # base-10 logarithm
print(np.log2(x))   # base-2 logarithm
print(np.sqrt(x))   # square root
```

Note: For real-valued results, logarithm inputs must be positive. Square-root inputs should be non-negative.

Sum and cumulative sum

```
x = np.array([10, 8, 16, 100])

print(np.sum(x))       # total sum
print(np.cumsum(x))    # running/prefix sum
```

Note: The notebook used np.cumulative_sum(). np.cumsum() is the widely used equivalent for a cumulative sum and works across a broader range of NumPy versions.

16. Comparison and logical functions

Comparison operators return Boolean arrays containing True or False for each element.

```
x = np.array([10, 8, 16, 100])
y = np.array([2, 3, 16, 5])

print(x > y)
print(x < y)
print(x >= y)
print(x <= y)
print(x == y)
print(x != y)
```

np.all() and np.any()

```
greater_than = x > y

print(np.all(greater_than))  # True only if every value is True
print(np.any(x == y))        # True if at least one value is True
```

Note: all() means every condition must be True. any() means at least one condition must be True.

17. Sorting

In-place sorting with sort()

Sorts the original array and changes it.

Syntax

```
array.sort()
```

Example

```
x = np.array([10, 8, 16, 100])
z = x.copy()
z.sort()

print(z)  # [  8  10  16 100]
print(x)  # original x remains unchanged because z is a copy
```

Copy sorting with np.sort()

Returns a sorted copy and leaves the original array unchanged.

Syntax

```
sorted_array = np.sort(array)
```

Example

```
x = np.array([10, 8, 16, 100])
sort_arr = np.sort(x)

print(sort_arr)
print(x)
```

Sorting a 2D array

axis=1 sorts each row; axis=0 sorts each column.

Syntax

```
np.sort(array, axis=1)  # row-wise
np.sort(array, axis=0)  # column-wise
```

Example

```
mat = np.array([[10, 3, 5],
                [8, 4, 9]])

print(np.sort(mat, axis=1))
print(np.sort(mat, axis=0))
```

18. Searching with np.where()

np.where() can return the positions where a condition is true, or build a new array by choosing values based on a condition.

```
x = np.array([10, 8, 16, 100])

# Return indexes where the condition is True
indexes = np.where(x == 8)
print(indexes)

# Keep values greater than 8; replace the rest with 0
result = np.where(x > 8, x, 0)
print(result)
```

2D searching

```
mat = np.array([[10, 3, 5],
                [8, 4, 9]])

indexes = np.where(mat > 8)
result = np.where(mat > 8, mat, 0)

print(indexes)
print(result)
```

np.argmax() and np.argmin()

```
x = np.array([10, 100, 8, 16, 100])

print(np.argmax(x))  # index of first maximum
print(np.argmin(x))  # index of first minimum
```

Note: If the maximum or minimum occurs more than once, argmax() and argmin() return the first matching index.

19. Counting and unique values

np.count_nonzero()

Count how many elements satisfy a condition (True values are counted as nonzero).

Syntax

```
np.count_nonzero(condition)
```

Example

```
a = np.array([10, 70, 45, 90, 20, 80])
count = np.count_nonzero(a > 60)
print(count)
```

Expected output

```
3
```

np.unique()

Return the sorted unique values in an array. With return_counts=True, also return how often each unique value appears.

Syntax

```
np.unique(array)
np.unique(array, return_counts=True)
```

Example

```
a = np.array([2, 1, 2, 3, 1, 2])

unique_values = np.unique(a)
unique_values, counts = np.unique(a, return_counts=True)

print(unique_values)
print(counts)
```

Expected output

```
[1 2 3]
[2 3 1]
```

20. Loading data and statistical functions

Load CSV or text data with np.genfromtxt()

genfromtxt() reads numeric data from a text file. delimiter specifies the separator; skip_header=1 skips the first line, often used for column headings.

```
# The file must exist in the notebook's working directory.
data = np.genfromtxt(
    "student_scores.csv",
    delimiter=",",
    skip_header=1
)

print(data)
print(data.shape)
```

Note: The uploaded notebook refers to student_scores.csv, but that data file was not included with the two uploaded notebooks. The exact output and column meanings therefore cannot be verified from the supplied materials.

Select a column and calculate statistics

```
# Select the first column as a 1D array
math_marks = data[:, 0]

print(math_marks)

print(np.max(math_marks))     # maximum
print(np.min(math_marks))     # minimum
print(np.mean(math_marks))    # average
print(np.median(math_marks))  # middle value
print(np.std(math_marks))     # standard deviation
```

Note: The notebook used data[:, :1], which keeps the selected column 2D. data[:, 0] selects it as a 1D array and is simpler for these calculations. If the first column is not Math marks in the CSV, select the correct column index after checking the file's column order.

Standard deviation

Standard deviation describes how spread out values are around their mean. A low standard deviation means values are closer to the mean; a high standard deviation means they are more spread out.

Correlation coefficient with np.corrcoef()

```
study_hours = np.array([2, 4, 5, 7, 8])
exam_scores = np.array([65, 75, 78, 88, 92])

data_pair = np.array([study_hours, exam_scores])
correlation = np.corrcoef(data_pair)

print(correlation)
```

Note: The result is a 2 × 2 correlation matrix. The diagonal values are 1. The off-diagonal values show the Pearson correlation between study hours and exam scores; values closer to 1 indicate a stronger positive linear relationship.

21. Basic linear algebra

Matrix multiplication with np.dot()

For matrix multiplication, the number of columns in the first matrix must equal the number of rows in the second matrix.

```
A = np.array([
    [1, 2, 3],
    [4, 5, 6]
])                      # shape (2, 3)

B = np.array([
    [7, 8],
    [9, 10],
    [11, 12]
])                      # shape (3, 2)

product = np.dot(A, B)   # result shape (2, 2)
print(product)
```

Note: The inner dimensions must match: (2, 3) dot (3, 2) produces a (2, 2) result.

Trace

```
print(np.trace(B))
```

Note: The trace is the sum of the main diagonal elements. For a non-square matrix, it sums the main diagonal entries that exist.

Determinant and matrix rank

```
sq_mat = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
])

det_value = np.linalg.det(sq_mat)
rank_value = np.linalg.matrix_rank(sq_mat)

print(det_value)
print(rank_value)
```

Note: This example has determinant 0 and rank 2 because its rows are linearly dependent. A determinant is defined for square matrices.

Another dot-product example

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

print(np.dot(A, B))
```

22. Quick revision table

Task	NumPy syntax
Create an array	np.array(data)
Dimensions / shape / type / count	arr.ndim / arr.shape / arr.dtype / arr.size
Zeros / ones / fixed value	np.zeros(shape) / np.ones(shape) / np.full(shape, value)
Random values	np.random.rand(...) / randint(...) / uniform(...)
Range / evenly spaced values	np.arange(...) / np.linspace(...) / np.logspace(...)
Reshape / flatten	arr.reshape(...) / arr.flatten() / np.ravel(...)
Join / transpose / split	np.concatenate(...) / arr.T / np.array_split(...)
Sum / cumulative sum	np.sum(arr) / np.cumsum(arr)
Conditional selection	arr[condition] / np.where(condition, x, y)
Sort / unique / count	np.sort(arr) / np.unique(arr) / np.count_nonzero(...)
Statistics	np.max / np.min / np.mean / np.median / np.std
Matrix operations	np.dot / np.trace / np.linalg.det / np.linalg.matrix_rank

23. Corrections and source limitations

The guide preserves the topics and learning level of the two notebooks while correcting issues that would otherwise cause errors or confusion.

- Boolean indexing: changed the mismatched arr variable to the intended array variable (arr2).
- Splitting: changed the invalid equal split of 10 items into 3 parts to np.array_split(); demonstrated np.split() with 9 items.
- Cumulative sum: used np.cumsum() as the broadly compatible cumulative-sum function instead of np.cumulative_sum().
- Statistics column: used data[:, 0] to select a 1D column, rather than data[:, :1], which keeps a 2D column.
- Dictionary and set conversions: clarified that sets are unordered and mixed key/value data may be converted to a common NumPy type.
- Slicing: explained why .copy() is used when edits should not affect the original array.
- Empty arrays: clarified that np.empty() does not initialize values.
- Concatenation: clarified the dimension requirements for axis=0 and axis=1.

Missing or unclear source information:

- student_scores.csv was referenced but not uploaded, so its exact columns, row count and sample statistical results are not available.
- Some notebook cells only displayed values interactively or had print statements commented out; the guide uses representative outputs only where they are deterministic.
- Random-array outputs are intentionally not fixed because they change on each run.